ILIAR – Compte rendu final

Ibrahim Abouakil - Ghiles Besseghir - Léo Loi

Janvier 2026

Table des matières

1 Introduction générale **2**

2 Mise à jour de la collecte de données **3**
2.1 Ajustements apportés à la collecte de données . . . 3
2.2 Impact attendu sur l'apprentissage . . . 3

3 Réseau de neurones : conception et apprentissage **4**
3.1 Conception du réseau de neurones . . . 4
3.1.1 Choix du modèle et clarification des architectures implémentées . . . 4
3.1.2 Formulation du problème . . . 4
3.2 Représentation des données . . . 4
3.2.1 Entrées du modèle . . . 4
3.2.2 Sortie du modèle . . . 5
3.3 Architecture du réseau convolutif (FancyCNN) . . . 5
3.3.1 Structure générale . . . 5
3.3.2 Extracteur de caractéristiques convolutif . . . 5
3.3.3 Réduction spatiale globale . . . 6
3.3.4 Couche de régression finale . . . 6
3.3.5 Justification des choix architecturaux . . . 6

4 Paramètres et stratégie d'entraînement **7**
4.1 Fonction de coût . . . 7
4.2 Optimisation . . . 7
4.3 Protocole d'entraînement et de validation . . . 7

5 Discussion et limites **7**

6 Architecture du contrôleur **8**
6.1 L'autopilote . . . 8
6.2 La téléopération . . . 8
6.3 Le multiplexeur . . . 8

7 Évaluation du contrôleur : **9**
7.1 Métriques de l'évaluateur . . . 9
7.1.1 La distance entre la voiture et le chemin à suivre . . . 9
7.1.2 La distance parcourue en avant par la voiture . . . 10
7.2 L'évaluation de notre contrôleur . . . 10

8 Conclusion **11**

1 Introduction générale

Dans le cadre du module ILIAR, ce projet vise à mettre en œuvre une chaîne complète de conduite autonome pour un véhicule simulé dans Gazebo. Le véhicule évolue sur un circuit et doit rester dans sa voie tout en parcourant la plus grande distance possible. Le système est développé autour de ROS2 et s'appuie sur une architecture modulaire composée de plusieurs nœuds dédiés à la simulation, la commande et l'évaluation.

L'objectif principal est d'apprendre un modèle capable de produire une commande de direction à partir d'un flux d'images embarquées. Plus précisément, le véhicule dispose de trois caméras (gauche, avant, droite) fournissant une perception visuelle de la route à chaque instant. Le système doit prédire l'angle de braquage associé afin de maintenir une trajectoire correcte.

Dans un premier temps, le véhicule est piloté manuellement en simulation à l'aide d'une manette grâce à un nœud de téléopération. Cette phase permet de générer des trajectoires de conduite servant de référence. Un nœud de collecte est ensuite utilisé pour enregistrer les données nécessaires à l'apprentissage. Il synchronise les images provenant des caméras du véhicule et les associe à la commande de direction appliquée à chaque instant, afin de constituer des jeux de données destinés à l'entraînement et à la validation.

Une fois les données collectées, un réseau de neurones est entraîné à l'aide de la bibliothèque PyTorch pour prédire l'angle de braquage à partir des images. Le modèle obtenu est ensuite intégré dans un nœud de contrôle automatique (autopilot), chargé de produire en temps réel les commandes de direction du véhicule en simulation. Un multiplexeur de commandes permet de basculer entre le contrôle automatique et la téléopération manuelle, afin de garantir une reprise de contrôle en cas de comportement instable. Les performances du système sont enfin évaluées en simulation, notamment à travers la capacité du véhicule à rester dans sa voie et à parcourir une distance significative sans sortir du circuit.

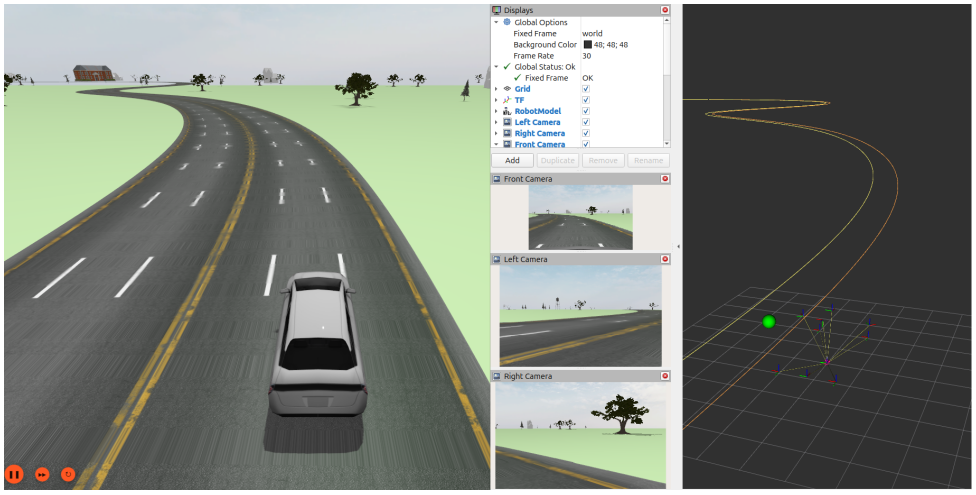


FIGURE 1 – Visualisation de la trajectoire du véhicule en simulation.

2 Mise à jour de la collecte de données

Lors du rapport intermédiaire, un premier jeu de données a été constitué en téléopération afin d'associer à chaque instant les images issues des trois caméras (gauche, avant et droite) à la commande de direction appliquée au véhicule. Les flux d'images étaient synchronisés de manière approximative afin de garantir la cohérence temporelle des échantillons collectés. Les données étaient ensuite stockées sous forme de fichiers compressés et organisées en blocs, puis un nettoyage manuel a été réalisé afin d'éliminer les séquences non exploitables tout en conservant des situations de conduite réalistes.

Les premiers entraînements du modèle et les tests en simulation ont toutefois mis en évidence certaines limites du jeu de données initial, notamment un manque de diversité dans certaines situations de conduite et une surreprésentation des trajectoires en ligne droite. Afin d'améliorer la qualité du jeu de données et d'obtenir un apprentissage plus robuste, la collecte a donc été mise à jour.

Dans la suite de cette section, nous présentons les modifications apportées à la collecte de données, les nouveaux jeux de données générés ainsi que l'impact attendu de ces changements sur les performances du modèle.

La mise à jour de la collecte de données ne modifie pas le protocole initial, mais apporte seulement quelques ajustements au dispositif existant.

2.1 Ajustements apportés à la collecte de données

Après les premiers essais d'apprentissage et les premiers tests en simulation, nous avons analysé le jeu de données utilisé. Cette analyse a montré que certaines situations de conduite pouvaient être davantage représentées pour améliorer le comportement du véhicule. Pour cette raison, quelques sessions de collecte supplémentaires ont été réalisées. Lors de ces sessions, la vitesse du véhicule a été légèrement augmentée par rapport à la première phase de collecte. Cela permet d'obtenir des situations où des braquages plus importants sont nécessaires, ce qui rend les commandes de direction plus cohérentes avec la dynamique du véhicule. Nous avons également fait attention à la répartition des situations de conduite. Les séquences correspondant aux virages et aux phases de correction ont été privilégiées, tandis que certaines séquences longues en ligne droite, moins informatives pour l'apprentissage, ont été réduites. La collecte a aussi été réalisée sur différents circuits afin d'introduire davantage de variété dans les types de virages rencontrés. Ces ajouts permettent d'enrichir le jeu de données existant sans remettre en cause le protocole de collecte défini lors du rapport intermédiaire.

2.2 Impact attendu sur l'apprentissage

Les premiers tests ont montré que le modèle entraîné avec le jeu de données initial se comportait correctement à très faible vitesse. En revanche, lorsque la vitesse du véhicule augmentait, le contrôleur devenait moins stable et avait tendance à dériver, rendant la conduite plus difficile à maîtriser. Les ajustements apportés à la collecte de données, et en particulier l'ajout de situations à vitesse légèrement plus élevée, visent à améliorer ce comportement. En exposant le modèle à des situations plus dynamiques lors de l'apprentissage, nous espérons obtenir des commandes de direction plus cohérentes lorsque le véhicule roule plus vite, et ainsi améliorer la stabilité globale du contrôleur en simulation.

3 Réseau de neurones : conception et apprentissage

3.1 Conception du réseau de neurones

3.1.1 Choix du modèle et clarification des architectures implémentées

Dans le cadre de ce projet, plusieurs architectures de réseaux de neurones ont été implémentées et évaluées, notamment :

— Un modèle linéaire (Linear).

— Un perceptron multi-couches (FFN – Feed-forward Network).

— Un réseau de neurones convolutif (CNN) nommé *FancyCNN*.

Les modèles linéaires et entièrement connectés constituent des baselines qui permettent de valider la chaîne d'apprentissage et de servir de points de comparaison. Toutefois, ces architectures ne permettent pas d'exploiter efficacement la structure spatiale des données visuelles, les images étant entièrement aplaties avant traitement.

Après analyse, le modèle retenu pour l'entraînement final et le déploiement est un réseau de neurones convolutif (CNN). Ce choix est motivé par sa capacité à extraire automatiquement des caractéristiques spatiales pertinentes à partir d'images, tout en limitant le nombre de paramètres et en améliorant la généralisation.

3.1.2 Formulation du problème

L'objectif du réseau de neurones est d'approximer une fonction de commande continue permettant de prédire la commande de direction d'un véhicule à partir de données visuelles multi-caméras.

Le problème est formulé comme une tâche de régression supervisée dans laquelle le réseau apprend une politique de contrôle à partir de démonstrations collectées lors de phases de téléopération.

Cette approche s'inscrit dans le cadre de l'apprentissage par imitation, où le modèle vise à reproduire le comportement d'un contrôleur expert.

Le modèle met en œuvre une approche end-to-end reliant directement la perception visuelle à l'action de conduite, sans étape intermédiaire de planification explicite.

3.2 Représentation des données

3.2.1 Entrées du modèle

Les observations d'entrée sont constituées de trois images RGB synchronisées issues de caméras disposées à gauche, à l'avant et à droite de la voiture.

Chaque image est :

— Redimensionnée à 128×128 pixels.

— Normalisée dans l'intervalle $[0, 1]$.

— Représentée sous la forme d'un tenseur de dimension $(3, 128, 128)$.

Les trois images sont concaténées le long de la dimension des canaux, ce qui conduit à une entrée tensorielle de dimension :

$$X \in \mathbb{R}^{9\times128\times128}$$

Cette représentation multi-vue permet d'enrichir l'information perceptive disponible et d'améliorer la robustesse du modèle face aux variations de trajectoire.

L'utilisation de trois caméras (gauche, centrale et droite) permet également de renforcer la robustesse du modèle face aux décalages latéraux du véhicule et de favoriser l'apprentissage de comportements de recentrage de la trajectoire, en exposant le réseau à des perspectives visuelles complémentaires.

3.2.2 Sortie du modèle

La sortie du réseau est un scalaire réel correspondant à la commande de direction du véhicule.

Aucune fonction d'activation n'est appliquée en sortie, conformément à la nature continue de la variable prédite.

3.3 Architecture du réseau convolutif (FancyCNN)

3.3.1 Structure générale

Le modèle *FancyCNN* est un réseau de neurones convolutif hiérarchique conçu pour transformer des observations visuelles brutes en une commande de direction continue.

Son architecture est organisée autour de trois blocs fonctionnels principaux :

1. Un extracteur de caractéristiques convolutif, chargé d'analyser les images d'entrée et d'extraire des représentations pertinentes.
2. Un mécanisme de réduction spatiale globale, visant à condenser l'information tout en favorisant la généralisation.
3. Une couche de régression finale, assurant la projection des caractéristiques extraites vers l'espace des commandes.

Cette organisation modulaire facilite l'interprétation du modèle ainsi que son intégration dans une architecture de contrôle temps réel.

3.3.2 Extracteur de caractéristiques convolutif

L'extracteur de caractéristiques constitue le cœur du réseau. Il repose sur une succession de blocs convolutifs hiérarchiques permettant de tirer parti de la structure spatiale des images multi-caméras.

Chaque bloc est composé de :

— Une convolution 3×3 avec un pas de 1 et un padding conservant la résolution spatiale.

— Une fonction d'activation ReLU, introduisant de la non-linéarité.

— Une normalisation par batch (*Batch Normalization*), stabilisant l'apprentissage et améliorant la convergence.

Afin de contrôler la complexité du modèle et de capturer des informations à différentes échelles, des convolutions de *downsampling* (2×2, stride 2) sont intercalées entre les blocs.

L'enchaînement des couches est le suivant :

— **Bloc 1 :**
Convolution 3×3 (stride 1, padding 1) : $9 \rightarrow 16$, suivie d'une activation ReLU et d'une normalisation par batch.
$\rightarrow$ Extraction de motifs visuels élémentaires à partir des images concaténées.

— **Bloc 2 (downsampling) :**
Convolution 2×2 avec un stride de 2 : $16 \rightarrow 32$, suivie d'une activation ReLU et d'une normalisation par batch.
$\rightarrow$ Réduction par un facteur 2 de la résolution spatiale et augmentation de la capacité de représentation.

— **Bloc 3 :**
Convolution 3×3 (stride 1, padding 1) : $32 \rightarrow 32$, suivie d'une activation ReLU et d'une normalisation par batch.
$\rightarrow$ Raffinement des caractéristiques intermédiaires.

— **Bloc 4 (downsampling) :**
Convolution 2×2 avec un stride de 2 : 32 → 64, suivie d'une activation ReLU et d'une normalisation par batch.
→ Capture de structures plus globales de la scène tout en réduisant la résolution spatiale.

— **Bloc 5 :**
Convolution 3×3 (stride 1, padding 1) : 64 → 64, suivie d'une activation ReLU et d'une normalisation par batch.
→ Consolidation des caractéristiques de haut niveau.

Cette progression hiérarchique permet au réseau de passer progressivement d'une représentation perceptive locale à une représentation abstraite exploitable pour la prise de décision.

3.3.3 Réduction spatiale globale

À l'issue des blocs convolutifs, les cartes de caractéristiques conservent une dimension spatiale résiduelle.

Un *Adaptive Average Pooling* est alors appliqué afin de projeter chaque carte de caractéristiques vers une représentation de taille 1×1.

Ce mécanisme présente plusieurs avantages :

— Réduction significative du nombre de paramètres.
— Limitation du sur-apprentissage.
— Invariance partielle aux translations.
— Indépendance vis-à-vis de la taille exacte des images d'entrée.

Il constitue ainsi un choix pertinent dans un contexte de conduite autonome, où les conditions visuelles peuvent varier.

3.3.4 Couche de régression finale

Les caractéristiques obtenues après le pooling global sont aplaties et transmises à une couche linéaire finale.

Cette couche assure la transformation de la représentation interne du réseau en une commande de direction scalaire.

L'absence de fonction d'activation en sortie permet de produire des valeurs continues sans contrainte artificielle sur leur amplitude, en accord avec la formulation du problème.

3.3.5 Justification des choix architecturaux

Les choix architecturaux retenus pour le modèle *FancyCNN* répondent aux contraintes spécifiques du problème étudié :

— Exploitation efficace de données visuelles multi-caméras.
— Compatibilité avec une exécution en temps réel.
— Compromis entre expressivité du modèle et capacité de généralisation.

L'association de convolutions hiérarchiques, de normalisation par batch et de pooling global permet d'obtenir un modèle robuste, stable à l'entraînement et peu sensible au sur-apprentissage.

Le modèle décrit correspond à une version du réseau initial. Il s'agit d'une configuration de départ fondée sur des choix d'architecture et de paramètres définis a priori. Par la suite, certains paramètres peuvent être ajustés lors des phases d'entraînement afin d'améliorer les performances, tout en conservant la même architecture et la même approche générale.

4 Paramètres et stratégie d'entraînement

4.1 Fonction de coût

La fonction de coût utilisée pour l'apprentissage du modèle est l'erreur quadratique moyenne (*Mean Squared Error* – MSE), définie par :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

où :

— y_i est la commande de direction réelle.

— $\hat{y}_i$ est la commande de direction prédite par le modèle.

— N est le nombre d'échantillons dans le batch.

Cette fonction de coût est particulièrement adaptée aux problèmes de régression continue et pénalise fortement les erreurs de prédiction importantes.

4.2 Optimisation

L'apprentissage du modèle est réalisé à l'aide de l'optimiseur *Adam*, avec les hyperparamètres suivants :

— Taux d'apprentissage : 1×10^{-3}.

— Régularisation L2 (*weight decay*) : 1×10^{-4}.

— Taille de batch : 128.

— Nombre d'époques : 40.

Ces paramètres offrent un compromis satisfaisant entre rapidité de convergence et stabilité de l'apprentissage.

4.3 Protocole d'entraînement et de validation

Le jeu de données est divisé en ensembles d'entraînement et de validation selon un ratio de 80% / 20%.

La perte sur l'ensemble de validation est suivie à chaque époque, et le modèle présentant la meilleure performance est sauvegardé automatiquement.

Ce protocole permet de limiter le sur-apprentissage et d'assurer une bonne capacité de généralisation.

5 Discussion et limites

Malgré de bonnes performances en simulation, le modèle présente certaines limites. En particulier, il ne prend pas explicitement en compte la dynamique temporelle de la conduite, chaque prédiction étant effectuée indépendamment à partir d'une observation visuelle instantanée.

Par ailleurs, les performances du modèle dépendent fortement de la qualité et de la diversité des démonstrations humaines utilisées pour l'apprentissage.

6 Architecture du contrôleur

Notre contrôleur peut être analysé en 3 parties distinctes :

6.1 L'autopilote

Tout d'abord, nous avons la partie autopilote où les données de la simulation sont envoyées au modèle puis ses sorties sont traitées puis envoyées comme commandes à la voiture.

Pour ce faire, nous avons créé un nœud autopilot qui charge un modèle donné sous la forme d'un fichier onnx.
Ce noeud souscrit à 3 topics :

- /audibot/left_camera/image_raw/compressed
- /audibot/front_camera/image_raw/compressed
- /audibot/right_camera/image_raw/compressed

Il récupère depuis ces topics les différentes images captées par les caméras de la voiture. Il synchronise les images puis transforme chacune d'elle en une matrice openCV.
On modifie le format des images et les concatènes pour qu'elles correspondent au format d'entrée du modèle.
On envoie cette donnée au modèle et on récupère la sortie qui permet entre autres de déterminer le prochain angle de braquage nécessaire.
On envoie ensuite cette valeur sur le topic /autopilot_steering et on envoie une valeur de throttle fixée à 0.1 sur le topic /autopilot_throttle

Nous avons décidé de publier sur les deux topics cités précédemment et non pas ceux directement écoutés par la simulation (/audibot/steering_cmd, /audibot/throttle_cmd) afin de faciliter les différentes manipulations du multiplexeur qui seront explicitées plus tard. Si seul l'autopilote est lancé, la redirection des topics est prise en charge par le fichier launch

6.2 La téléopération

Le noeud de téléopération permet d'écouter les différents inputs rentrés sur une manette et d'envoyer les commandes associées à la simulation.

Dans ce noeud, nous mettons en place divers calculs sur les inputs reçus de la manette pour qu'ils puissent correspondent aux données attendues par la simulation et surtout pour qu la conduite soit possible par un humain (par exemple, faire en sorte qu'un léger appuie sur le bouton d'accélération se transmette en une accélération douce et non forte.)
En plus de cela, nous avons mis en place un deadman button, i.e. un bouton qui tant qui doit être pressé pour que les commandes sont envoyées à la simulation. Ainsi, si ce bouton n'est pas pressé, aucune donnée n'est publiée sur les différents topics sur lesquels ce nœud peut publier.

6.3 Le multiplexeur

Pour faire en sorte que l'autopilote et la téléopération puissent coexister, nous devons mettre en place un multiplexeur qui enverra à la simulation les données nécessaires.

Tout d'abord, le multiplexeur doit récupérer les informations envoyées par la téléopération et l'autopilot. Le multiplexeur souscrit donc aux différents topics sur lesquels publient la téléopération et l'autopilote.
On voit ici pourquoi on a décidé de séparer les topics des nœuds précédents. En effet, il est important pour

que le multiplexeur puisse déterminer quel nœud écouter et qu'il puisse déterminer quelles informations vient de quel nœud.

L'objectif de ce multiplexeur va être de laisser l'autopilot contrôler la voiture mais si quelqu'un souhaite récupérer le contrôle de la voiture, il le peut grâce à la manette et au nœud de téléopération.
Pour ce faire, on récupère la clock du nœud et on relève le temps de la dernière instruction envoyée par le nœud de téléopération. Si la durée entre cette commande et le temps actuel est inférieur à 0.5 secondes (500 000 000 nanosecondes) les commandes de l'autopilote sont ignorées. Sinon ce sont les commandes de l'autopilote qui sont prisent en compte et envoyées à la simulation.

7 Évaluation du contrôleur :

7.1 Métriques de l'évaluateur

Afin d'évaluer si le contrôleur, et surtout sa partie d'autopilote, décrit précédemment est efficace, nous avons mis en place plusieurs métriques décrivant son efficacité.

7.1.1 La distance entre la voiture et le chemin à suivre

Tout d'abord, nous avons cherché à calculer l'écart que pouvait avoir la voiture par rapport au chemin idéal à suivre.

Pour cela, il fallait commencer par récupérer le chemin idéal. Nous avons donc créé le node path_publisher qui récupère le fichier monde de gazebo et qui le parse pour en récupérer la route. Une fois ce chemin récupéré, il est publié sur le topic /road_path sous le format nav_msgs/Path

Après sa publication, le chemin est récupéré par le nœud dist_to_path.
Ce dernier prend les deux points du chemin les plus proche de la voiture et calcule la distance entre le segment composé de ces deux points et la voiture.

Maintenant que nous connaissons en tout temps la distance de la voiture au chemin, nous pouvons commencer à calculer des métriques d'évaluation de notre contrôleur.
Nous avons mis en place le calcule de 3 métriques différentes :

- **Le minimum :** Cette métrique, comparée avec le maximum, nous permet notamment de savoir si le contrôleur subit de grande variation (possibilité accrue que le contrôleur slalom) ou s'il reste constant à une distance constante de la trajectoire.
- **Le maximum :** Il nous permet de savoir à quel point la voiture à pu s'éloigner de la trajectoire voulue. En condition réelle, cette valeur peut s'avérer très utile afin de vérifier que la voiture ne dépasse pas un certain seuil qui, si dépassé, pourrait être dangereux pour les autres usagers.
- **La moyenne :** Elle permet de déterminer si le contrôleur ne fait pas ou peu de grosses erreurs/ Il faut cependant être vigilant car cette métrique étant une moyenne non pondérée, certaines erreurs qui peuvent être dangereuses en situation réelles peuvent être "annulée" par une conduite correcte surtout s'il y a beaucoup d'échantillons.

Ces métriques nous permettent d'avoir un aperçu global de l'efficacité de notre contrôleur mais elles ont aussi des faiblesses qu'il faut prendre en compte. Elles sont efficaces pour régler notre contrôleur mais il nous faudrait des métriques plus précises pour évaluer correctement notre contrôleur.

7.1.2 La distance parcourue en avant par la voiture

Afin de pouvoir porter une étude sur les métriques citées précédemment, nous avons mis en place le calcul de la distance parcourue en avant par le véhicule le long du chemin.
L'ajout de cette métrique permet de mettre un sens sur les 3 précédentes. En effet, avoir une de très bon minimum, maximum et moyenne sur une distance très courte n'a pas vraiment de sens et n'est pas représentatif des vraies performances du contrôleur. Il est donc important de les liées à une valeur de distance qui permettra de confirmer la qualité du contrôleur décrite par les autres métriques

Tout d'abord, il nous fallait calculer la distance parcourue par la voiture sans se préoccuper de la notion de "en avant". Pour se faire :

1. On récupère le chemin que la voiture doit suivre, représenté par un polygone, et on projette sur l'arête la plus proche la position de la voiture. Pour cela, on récupère la position de la voiture et pour chaque arête du polygone, on utilise la fonction project_point_on_segment de la bibliothèque geom2D. On prend ensuite la norme de la différence entre la position de la voiture et le point projeté. Ce qui nous donne la distance entre la voiture et le segment. Il ne nous reste plus qu'à prendre la valeur minimum.
2. On calcule la distance entre la dernière position valide et la position courante et on l'ajoute à la distance parcourue jusqu'à maintenant. La position courante devient alors la dernière position valide et on répète l'opération

La distance parcourue est ensuite obtenue en mesurant le déplacement entre deux positions successives de ce point projeté. Ce déplacement n'est ajouté à la distance totale que lorsque la voiture se déplace dans le sens du parcours ignorant ainsi tout mouvement de recul ou oscillation sur le long du parcours. Dans le cas d'un parcours fermé la transition entre la fin et le début du parcours est également prise en compte afin de garantir une mesure continue de la distance parcourue.

7.2 L'évaluation de notre contrôleur

Afin d'évaluer notre contrôleur, nous avons lancé la simulation avec l'autopilot et l'avons laissé rouler quelques temps tout en relevant les différentes métriques citées plus haut. Nous avons obtenus les résultats suivants obtenus dans la Table 1

Distance parcourue	Minimum	Moyenne	Maximum
30	0.23	0.89	1.75
50	0.06	0.67	1.75
100	0.06	0.51	1.75
200	0.06	0.78	2.51
500	0.005	1.76	4.99

TABLE 1 – Evolution des métriques le long de la simulation

On peut voir dans ce tableau que notre contrôleur semble le plus efficace avant d'atteindre les 100 unités de distance. Au delà la distance moyenne au chemin et la distance maximale augmente grandement. On peut expliquer cela notamment par la forme du circuit choisi (circuit1.sdf). Lorsque la distance atteint 100, le circuit a des virages plus serrés et qui s'enchaînent plus. Notre modèle semble donc correct mais il n'est pas très robuste à la présence répétée de virages ou de à la présence de virages serrés.

8 Conclusion

Dans ce rapport, nous avons donc explicité les différentes étapes de la mise en place d'un modèle visant à produire une commande de direction à partir d'un flux d'images.
Tout d'abord, nous avons précisé notre collecte de donnée et varié les situations pour essayer de généraliser l'apprentissage du modèle.
Ensuite, nous avons explicité les choix que nous avons fait concernant la structure du réseau de neurone utilisé pour la création du modèle ainsi que les raisons pour lesquels nous avons fait ces choix.
Après cela, nous avons présenter les différentes stratégies d'entraînement que nous avons pu mettre en place pour rendre aussi efficace que possible notre modèle.
Enfin, nous avons discuté du contrôleur mis en place pour faire le lien entre le modèle et la simulation et nous avons pu évaluer ses résultats sur une simulation.

Nous avons pu voir que notre modèle semblait efficace sur les lignes droites et les virages larges mais semble avoir du mal à rester dans sa voie en présence de virages serrés ou répétés.
Si nous avions plus de temps, nous aurions pu mettre en place diverses méthodes pour évaluer d'où viennent ces imperfections. Par exemple, nous pourrions essayer de garder la base de données que nous avons mais de modifier le réseau de neurones. ou nous pourrions aussi garder le réseau de neurone actuel et effectuer des apprentissages sur d'autres types de données (plus de virage, ...)